



北京邮电大学
Beijing University of Posts and Telecommunications

BUPT Compilers Lab2 2025Fall

Name:

Student ID:

Class Number:

Container Number:

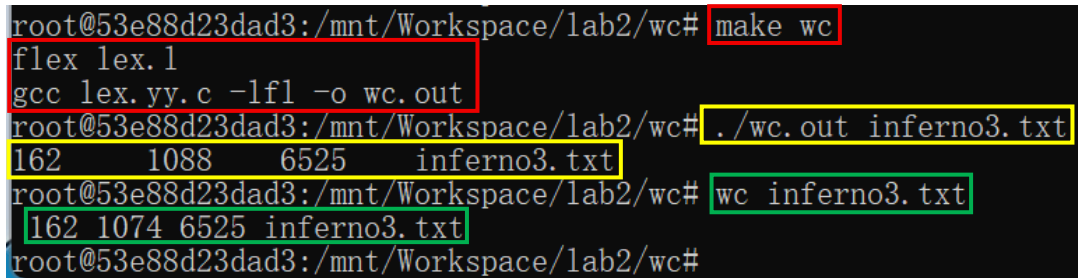
1 Running the wc program example

Make the wc target, run it and compare with system's wc.

The commands were used:

```
make wc
./wc.out inferno3.txt
wc inferno3.txt
```

1.1 The screenshot of the compilation and execution results



```
root@53e88d23dad3:/mnt/Workspace/lab2/wc# make wc
flex lex.l
gcc lex.yy.c -lfl -o wc.out
root@53e88d23dad3:/mnt/Workspace/lab2/wc# ./wc.out inferno3.txt
162      1088      6525      inferno3.txt
root@53e88d23dad3:/mnt/Workspace/lab2/wc# wc inferno3.txt
162 1074 6525 inferno3.txt
root@53e88d23dad3:/mnt/Workspace/lab2/wc#
```

Figure 1: Screenshot of the compilation and execution results

1.2 Explanation of the output of running this wc program is different from the wc command on Linux systems

Linux's wc treats a sequence of characters separated by spaces, tabs, or newlines as a word; while the example's wc treats a sequence of letters separated by other characters as a word.

2 Flex Exercises-identifiers

Make the identifiers target and run it.

The commands were used:

```
make idcount
./idcount.out ./test.c
```

2.1 I found that the line numbers in the output do not match the line numbers of the tokens

```

root@53e88d23dad3:/mnt/Workspace/lab2# cd identifiers
root@53e88d23dad3:/mnt/Workspace/lab2/identifiers# make idcount
flex lex.l
gcc lex.yy.c -lfl -o idcount.out
root@53e88d23dad3:/mnt/Workspace/lab2/identifiers# ./idcount.out ./test.c
line 0: int
line 0: main
line 0: int
line 0: a
line 0: int
line 0: BBA
line 0: a_
line 0: int
line 0: _
line 0: int
line 0: a
line 0: if
line 0: a0
line 0: _
line 0: b0
line 0: else
line 0: _
line 0: b0
line 0: while
line 0: b
line 0: b1
line 0: b0
line 0: b2
line 0: char
line 0: c
line 0: a
line 0: return
There are 27 occurrences of valid identifiers
root@53e88d23dad3:/mnt/Workspace/lab2/identifiers#

```

Figure 2: Screenshot of the compilation and execution results

2.2 I solved it with Flex in the lab session

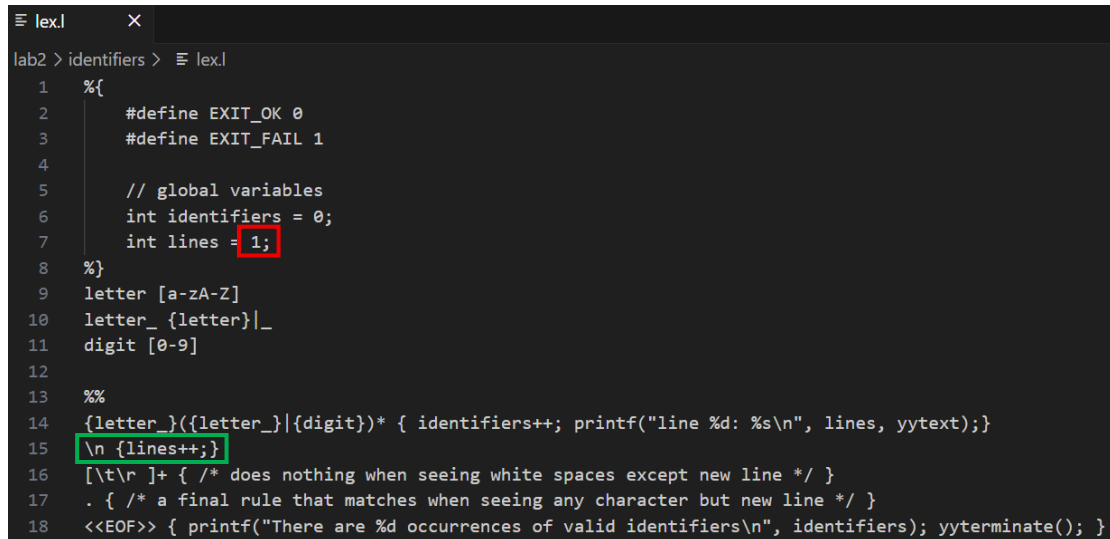
2.2.1 My code

Two changes are made to the lex.l file.

```

lex.l
lab2 > identifiers > lex.l
1  %{
2      #define EXIT_OK 0
3      #define EXIT_FAIL 1
4
5      // global variables
6      int identifiers = 0;
7      int lines = 0;
8  %}
9  letter [a-zA-Z]
10 letter_ {letter}|_
11 digit [0-9]
12
13 %%
14 {letter_}({letter_}|{digit})* { identifiers++; printf("line %d: %s\n", lines, yytext); }
15 \n {}
16 [\t\r]+ { /* does nothing when seeing white spaces except new line */ }
17 . { /* a final rule that matches when seeing any character but new line */ }
18 <<EOF>> { printf("There are %d occurrences of valid identifiers\n", identifiers); yyterminate(); }

```



```
lab2 > identifiers > lex.l
1  %{
2      #define EXIT_OK 0
3      #define EXIT_FAIL 1
4
5      // global variables
6      int identifiers = 0;
7      int lines = 1;
8  %}
9  letter [a-zA-Z]
10 letter_ {letter}|_
11 digit [0-9]
12
13 %%
14 {letter_}({letter_}|{digit})* { identifiers++; printf("line %d: %s\n", lines, yytext);}
15 \n {lines++;}
16 [\t\r ]+ { /* does nothing when seeing white spaces except new line */ }
17 . { /* a final rule that matches when seeing any character but new line */ }
18 <<EOF>> { printf("There are %d occurrences of valid identifiers\n", identifiers); yyterminate(); }
```

Figure 3: Two changes are made to the lex.l file.

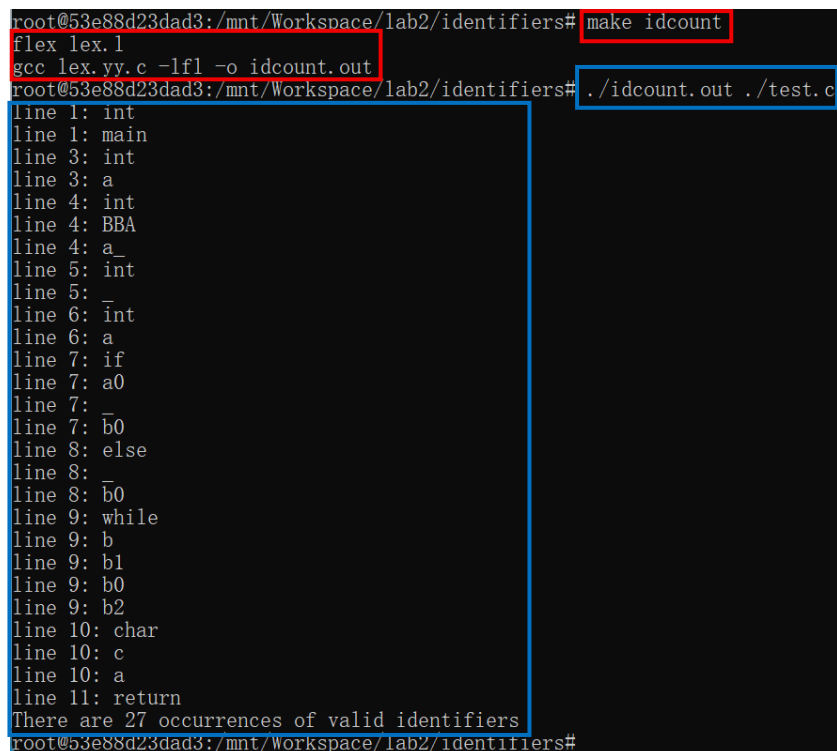
The changed code can be found at lex.l.

1. Change Line 7 to `int lines = 1;` ; it makes the initial line 1 as we count lines from 1.
2. Change Line 15 to `\n {lines++;}` ; it means each time the program detects `\n` , the variable line

2.2.2 The principles of key parts of my code

The initial value of lines should be 1, and when changing lines, the value of lines should be increased by 1 (self-incrementing).

2.2.3 The screenshot of the compilation and execution results



```
root@53e88d23dad3:/mnt/Workspace/lab2/identifiers# make idcount
flex lex.l
gcc lex.yy.c -lfl -o idcount.out
root@53e88d23dad3:/mnt/Workspace/lab2/identifiers# ./idcount.out ./test.c
line 1: int
line 1: main
line 3: int
line 3: a
line 4: int
line 4: BBA
line 4: a_
line 5: int
line 5: _
line 6: int
line 6: a
line 7: if
line 7: a0
line 7: _
line 7: b0
line 8: else
line 8: _
line 8: b0
line 9: while
line 9: b
line 9: b1
line 9: b0
line 9: b2
line 10: char
line 10: c
line 10: a
line 11: return
There are 27 occurrences of valid identifiers
root@53e88d23dad3:/mnt/Workspace/lab2/identifiers#
```

Figure 4: Screenshot of the compilation and execution result (change)

3 Flex Exercise -ipaddr

Validate IP Address.

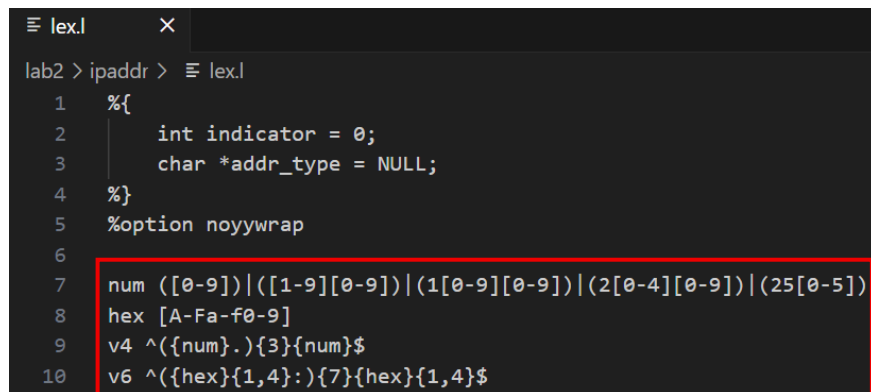
Solve it with Flex in the lab session and use ip_test.py to verify the results. Try to pass them!

The commands were used:

```
make ipaddr
python3 ip_test.py
```

3.1 My code

Four lines (7~10) of code were added to lex.l file.



```
lex.l
lab2 > ipaddr > lex.l
1  %{
2      int indicator = 0;
3      char *addr_type = NULL;
4  %}
5  %option noyywrap
6
7  num ([0-9])|([1-9][0-9])|(1[0-9][0-9])|(2[0-4][0-9])|(25[0-5])
8  hex [A-Fa-f0-9]
9  v4 ^({num}.){3}{num}$
10 v6 ^({hex}{1,4}:{7}{hex}{1,4}$
```

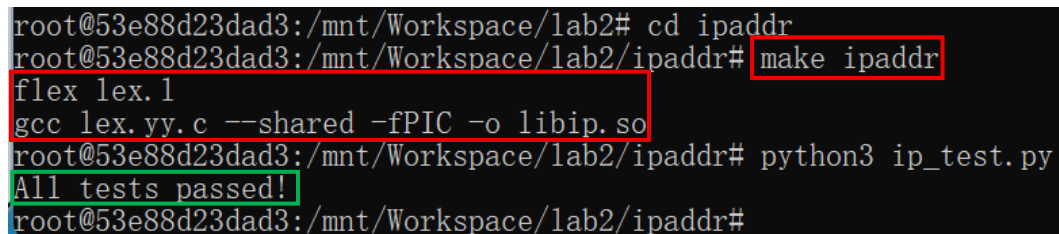
Figure 5: The code I added

The changed code can be found at lex.l.

3.2 The principles of key parts of my code

In this part, num defines the pattern of decimal numbers from 0 to 255. hex defines the pattern of hexadecimal numbers from 0 to f. v4 defines the pattern of an IPv4 address. v6 defines the pattern of an IPv6 address.

3.3 The screenshot of the compilation and execution results



```
root@53e88d23dad3:/mnt/Workspace/lab2# cd ipaddr
root@53e88d23dad3:/mnt/Workspace/lab2/ipaddr# make ipaddr
flex lex.l
gcc lex.yy.c --shared -fPIC -o libip.so
root@53e88d23dad3:/mnt/Workspace/lab2/ipaddr# python3 ip_test.py
All tests passed!
root@53e88d23dad3:/mnt/Workspace/lab2/ipaddr#
```

Figure 6: The screenshot of the compilation and execution results